

Large Language Models

Tirtharaj Dash

Department of CS & IS
BITS Pilani, Goa Campus, India
[Homepage](#)

August 2026

Where we are

The last two lectures (on “Intelligent Agents”) gave us the agent abstraction

$$f : P^* \rightarrow A.$$

Many current agentic systems put an LLM at their centre, surrounded by memory, tools and a planner. One slide in that lecture said:

By itself, an LLM is primarily a sequence prediction model.

Today we will discuss what that model actually is, and how one is built (albeit at an abstract level).

We plan to discuss:

1. What is a language model, mathematically?
2. What does *large* buy us?
3. How do we achieve task-solving behaviour?

We treat the network itself as a black box. Its internals — attention and the Transformer — will be discussed next week.

On terminology: a *foundation model* is any model pretrained on broad data at scale and then adapted to many downstream tasks. A large language model is the text case. [Bommasani et al. 2021]

A language model is a distribution over text

Fix a finite vocabulary V . A language model assigns a probability to every finite sequence of tokens:

$$P(x_1, x_2, \dots, x_T), \quad x_t \in V.$$

We cannot store this as a table. Why?

A language model is a distribution over text

With $|V| = 50,000$ and $T = 10$ there are

$$50,000^{10} \approx 10^{47}$$

possible sequences.

So we need a factorisation, and a function that computes each factor.

The chain rule does the work

For any joint distribution,

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}).$$

This is exact. No independence assumption has been made.

One hard problem has become T easier problems, each of the same shape:

given the tokens so far, put a distribution on the next one.

The chain rule does the work

The classical route was to truncate the history,

$$P(x_t \mid x_{<t}) \approx P(x_t \mid x_{t-n+1}, \dots, x_{t-1}),$$

which gives n -gram models. Counting works, but the number of contexts grows as $|V|^{n-1}$, so most contexts are never observed.

The **neural** alternative keeps the full history and replaces counting with a learned function

$$f_\theta : V^* \rightarrow \Delta(V),$$

where $\Delta(V)$ is the set of distributions over the vocabulary. Contexts that were never seen still receive sensible predictions, because similar contexts have similar representations.

Next-token prediction is a classification problem

This is the setting from our week-1 lecture, with one change: the number of *classes*.

The network maps a context to a vector of logits, one per vocabulary entry:

$$\mathbf{z} = f_{\theta}(x_{<t}) \in \mathbb{R}^{|V|}, \quad p_i = \frac{e^{z_i}}{\sum_{j=1}^{|V|} e^{z_j}}.$$

Softmax over 50,000 classes rather than 3. The loss is the same cross-entropy, and the gradients flow back in the same way.

The training objective

For a sequence $x_1, \dots, x_T$,

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}).$$

Minimising this is maximising the likelihood of the training text.

There are no human-supplied labels here. The label at position t is the token that actually occurs at position t . This is called *self-supervised* learning.

Data supply is measured in trillions of tokens rather than in annotator-hours.

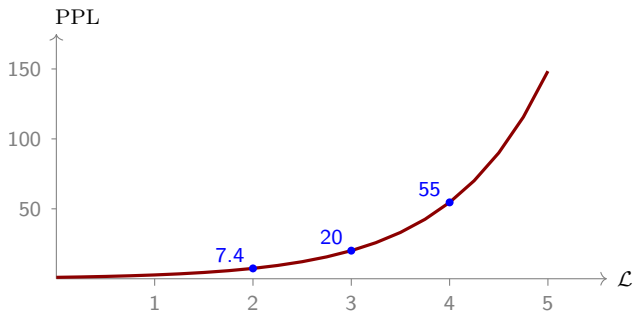
Raw loss values in nats lack an intuitive scale. To make results interpretable, LM studies report Perplexity

$$\text{PPL} = \exp(\mathcal{L}),$$

which maps directly to the vocabulary scale.

Under this metric, uniform random guessing yields a PPL equal to the vocabulary size ($|V|$), while a perfect model yields 1.

Perplexity



Read it as an *effective branching factor*: how many equally likely tokens the model is choosing among. One nat from 4 to 3 removes 34 effective choices; the same nat from 2 to 1 removes under 5.

Where does the training signal come from?

Language models do not require manually assigned labels. In a sequence of text, each token provides the target for predicting what comes next. Thus, ordinary text automatically becomes training data.

For a context window

$$x_1, x_2, \dots, x_{n_{\text{ctx}}},$$

the model is trained to make the predictions

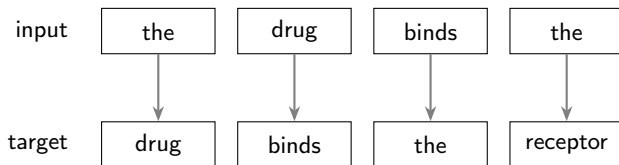
$$x_1 \rightarrow x_2, \quad (x_1, x_2) \rightarrow x_3, \quad \dots, \quad (x_1, \dots, x_{n_{\text{ctx}}-1}) \rightarrow x_{n_{\text{ctx}}}.$$

The window is then moved through the corpus to generate further training examples.

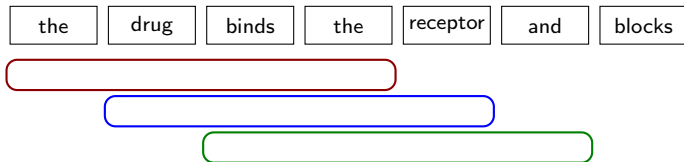
Where does the training signal come from?

- ▶ Every token after the first supplies a training signal. Hence, a corpus containing N tokens provides roughly N next-token prediction targets.
- ▶ To predict the next token well, the model must learn recurring patterns of grammar, facts, arithmetic, code, and discourse—not because these are explicitly labelled, but because they help make the next token more predictable.

Constructing the training pairs



The target row is the input row shifted one position left.



example 1

example 2

example 3

The window slides one token at a time over the whole corpus.

From text to token embeddings

Subword tokenisation

A language model does not process text directly. A *tokeniser* converts text into a sequence of integers:

$$\text{text} \longrightarrow (t_1, t_2, \dots, t_n), \quad t_i \in \{1, \dots, |V|\}.$$

Two natural choices are unsatisfactory:

- ▶ **Word-level tokens:** the vocabulary becomes enormous, unseen words cause problems, and related forms such as `bind`, `binds`, and `binding` receive unrelated identifiers.
- ▶ **Character-level tokens:** the vocabulary is small, but sequences become much longer and the model must spend substantial capacity reconstructing words from characters.

From text to token embeddings

Subword tokenisation provides a compromise: frequent words remain whole, while rare words are decomposed into reusable pieces:

binding $\longrightarrow$ bind + ing.

From text to token embeddings

How is the vocabulary constructed?

A widely used method is **byte-pair encoding (BPE)**:

1. Begin with a small base vocabulary—characters or bytes.
2. Count all adjacent token pairs in the training corpus.
3. Merge the most frequent pair into a new token.
4. Repeat until the desired vocabulary size is reached.

For example, repeated merges might produce

$$l + o \longrightarrow lo, \quad lo + w \longrightarrow low.$$

Thus, common strings receive short representations, while rare strings are represented using smaller pieces. With byte-level tokenisation, every input can be represented, so there is no unknown-word problem.

From text to token embeddings

What does tokenisation cost us?

- ▶ *Different words have different costs.* A common word may use one token, whereas a rare word may require several.
- ▶ *Computation is measured in tokens, not words.* Tokenisation therefore affects context length, memory, training cost, and generation speed.
- ▶ *Specialised notation may fragment badly.* SMILES strings, gene identifiers, equations, and source code can be divided at positions that have little scientific meaning.
- ▶ *Token boundaries affect what the model can easily learn.* If a word is stored as one token, the model does not directly receive its individual letters. This helps explain why spelling, counting letters, and other character-level tasks can be unexpectedly difficult.

From text to token embeddings

GPT-2 uses byte-level BPE with

$$|V| = 50,257.$$

Radford et al., 2019.

From text to token embeddings

From token identifiers to vectors

Token identifiers are arbitrary labels: token 317 is not numerically “closer” to token 318 than to token 9000. The model therefore converts each identifier into a learned vector.

The embedding table is

$$E \in \mathbb{R}^{|V| \times d},$$

where the i -th row,

$$E_{i,:} \in \mathbb{R}^d,$$

is the embedding of token i . In practice, this is simply a table lookup:

$$t_i \longrightarrow E_{t_i,:}.$$

These vectors are learned jointly with all the other model parameters.

From text to token embeddings

Embeddings are a substantial part of the model

For GPT-2 Small,

$$|V| = 50,257, \quad d = 768.$$

Hence, the token-embedding table contains

$$50,257 \times 768 = 38,597,376$$

parameters—about 39 million parameters in this table alone.

From text to token embeddings

After lookup, the sequence

$$(t_1, t_2, \dots, t_n)$$

has become a sequence of vectors

$$(E_{t_1,:}, E_{t_2,:}, \dots, E_{t_n,:}).$$

But token embeddings identify *what* the tokens are, not *where* they occur. Positional information must therefore be supplied separately.

The context window

When predicting the next token, the model can attend to at most n_{ctx} tokens:

$$p(x_t \mid x_{t-n_{\text{ctx}}}, \dots, x_{t-1}).$$

This context includes the prompt, conversation history, retrieved material, and usually the tokens generated so far.

The context window

GPT-2 used

$$n_{\text{ctx}} = 1024,$$

whereas some recent systems support around one million tokens—roughly $1000\times$ as many.

The context window is the model's working memory.

The context window

The model may contain general knowledge in its parameters, but information specific to the current task must be available in the active context.

[Radford et al., 2019]

What happens when the information does not fit?

A system must decide what to keep, retrieve, compress, or discard:

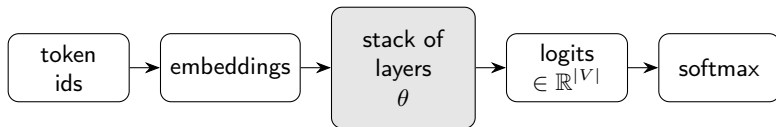
- ▶ **Retrieve:** find the most relevant external information and insert it into the prompt. Retrieval-augmented generation
- ▶ **Compress:** replace older or lengthy material with a shorter summary. Context compression
- ▶ **Store and recall:** save useful information externally and retrieve it when it becomes relevant again. Agent memory

The context window

External memory expands what the *system* can store, not what the *model* can attend to. A stored fact becomes usable only after it is retrieved into the active context.

A larger context window raises the capacity of the model's working memory; context management determines which information occupies it.

The model as a black box



The model as a black box

Everything discussed so far is independent of the architecture inside the grey box: it could be a recurrent network, a convolutional network, or a Transformer.

Transformers became dominant because they process all positions in parallel during training and allow information to interact directly across distant positions through self-attention. [Vaswani et al., NeurIPS 2017]

Scaling language models

Model	Parameters	Training data
GPT-2 (2019)	1.5 B	~40 GB of text
GPT-3 (2020)	175 B	300 B tokens
Chinchilla (2022)	70 B	1.4 T tokens
Llama 3 (2024)	405 B	~15 T tokens

The training objective remained essentially the same: predict the next token. What changed was the scale of the model, the training data, and the computation.

Radford et al., 2019; Brown et al., 2020; Hoffmann et al., 2022; Meta AI, 2024

Scaling language models

Parameters and data must scale together

Scaling laws show that performance improves predictably as both the number of parameters N and the number of training tokens D increase.

A widely used compute-optimal rule of thumb is

$$D \approx 20N.$$

Thus, a model with 70 billion parameters should be trained on roughly 1.4 trillion tokens.

GPT-3 had many parameters but relatively little training data. Chinchilla showed that a smaller, better-trained model could outperform much larger models.

[Kaplan et al., 2020; Hoffmann et al., 2022]

In-context learning

GPT-3 showed that a model can infer a task from examples placed in the prompt and perform it immediately—without any additional training. [Brown et al., NeurIPS 2020]

Example: English-to-French translation

Zero-shot	Translate to French: cheese →
	sea otter → loutre de mer
Few-shot	plush giraffe → girafe en peluche
	cheese →

The examples communicate both the task and the desired output format.

Is the model really learning?

In-context learning

Not in the usual training sense:

- ▶ no gradients are computed,
- ▶ no parameters are updated,
- ▶ the change in behaviour lasts only within the current context.

The examples condition the model to recognise and continue the demonstrated input–output pattern. This ability is called **in-context learning**.

In-context learning

Training changes the model's parameters;
prompting changes its current behaviour.

Three stages of training an assistant

1. **Pretraining.** Next-token prediction on a very large corpus. The result is a *base model*: it continues text.
2. **Supervised fine-tuning.** Further training on (instruction, response) pairs written by people. The result follows instructions.
3. **Preference alignment.** Human comparisons between candidate responses are turned into a training signal, by reinforcement learning or by direct preference optimisation. [Ouyang et al., NeurIPS 2022]

We have a dedicated lecture for these.

Three stages of training an assistant

The asymmetry is worth noting. Stage 1 consumes almost all of the compute and essentially all of the data. Stages 2 and 3 are comparatively tiny, and yet they account for nearly everything a user notices about the system's behaviour.

Example

Prompt: List three uses of aspirin.

- ▶ A *base* model may well continue with List three uses of paracetamol. — a plausible next line in a document that contains the prompt.
- ▶ An *instruction-tuned* model answers the question.

Answering rather than continuing is a trained behaviour. It is not a property of next-token prediction.

Decoding is a separate decision

The model gives us $p(\cdot \mid x_{<t})$. Turning that into text requires a rule, and the rule is not part of the model.

Example

Five candidate continuations of the cat sat on the, with logits $z = (3.0, 2.0, 1.0, 0.5, 0.0)$:

	mat	sofa	floor	roof	and
$T = 1.0$	0.612	0.225	0.083	0.050	0.030
$T = 0.5$	0.860	0.116	0.016	0.006	0.002
$T = 2.0$	0.403	0.244	0.148	0.115	0.090

Temperature rescales the logits before the softmax:

$$p_i \propto \exp(z_i/T).$$

Decoding is a separate decision

Greedy. Take $\arg \max_i p_i$. Deterministic; tends to repeat itself.

Top- k . Keep the k most probable tokens and renormalise. With $k = 2$ at $T = 1$:

mat : 0.731, sofa : 0.269.

Top- p (nucleus). Keep the smallest set whose probability mass reaches p . The cumulative masses above are 0.612, 0.837, 0.919, 0.970, 1.000, so $p = 0.9$ keeps three tokens:

0.665, 0.245, 0.090.

[Holtzman et al., ICLR 2020]

Unlike top- k , the size of the retained set adapts to the shape of the distribution.

Decoding is a separate decision

Two practical points:

- ▶ $T \rightarrow 0$ approaches greedy decoding. Large T flattens the distribution towards uniform.
- ▶ The same weights under two decoding settings behave like two different systems. Decoding parameters belong in the methods section of any experiment you report.

What the model does not have

- ▶ **Any notion of truth.** It was trained to be probable, not correct. A fluent false statement is a low-loss statement if the training distribution supports it.
- ▶ **Current knowledge.** Whatever it knows was fixed when training ended.
- ▶ **Memory.** Nothing persists between calls unless something outside the model stores it.
- ▶ **Actions.** It emits tokens. It cannot query a database, run a program or book a flight.
- ▶ **Exact computation.** Arithmetic, counting and character-level manipulation are unreliable, partly for the tokenisation reason given earlier.

What the model does not have

Compare that list with the components of a modern agent from the previous lecture:

Limitation	Component that addresses it
Stale or missing knowledge	Retrieval
No persistence	External memory
No actions	Tools and APIs
No verification	Critic, reflection
Domain mismatch	Fine-tuning

The agent architecture is not decoration around a language model. Each part of it exists because of a specific thing the model cannot do. Each row is a lecture later in this course.

Take-home messages

1. A language model is a distribution over token sequences, factorised by the chain rule into next-token predictions.
2. Training is ordinary cross-entropy classification over the vocabulary. The labels come free from the text, which is what allows the scale.
3. Text reaches the model as subword token ids and then as learned embeddings. Tokenisation quietly determines cost, context use and several failure modes.
4. *Large* refers to parameters, data and compute together, and the ratio between them matters as much as their size.
5. Instruction-following and helpfulness are added after pretraining. They are not consequences of next-token prediction.

References and further reading

- ▶ Bengio, Ducharme, Vincent, Jauvin. A neural probabilistic language model. *JMLR*, 3:1137–1155, 2003.
- ▶ Sennrich, Haddow, Birch. Neural machine translation of rare words with subword units. *ACL*, 2016.
- ▶ Vaswani et al. Attention is all you need. *NeurIPS*, 2017.
- ▶ Radford, Wu, Child, Luan, Amodei, Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019.
- ▶ Holtzman, Buys, Du, Forbes, Choi. The curious case of neural text degeneration. *ICLR*, 2020.
- ▶ Brown et al. Language models are few-shot learners. *NeurIPS*, 2020.
- ▶ Kaplan et al. Scaling laws for neural language models. arXiv:2001.08361, 2020.
- ▶ Bommasani et al. On the opportunities and risks of foundation models. arXiv:2108.07258, 2021.
- ▶ Hoffmann et al. Training compute-optimal large language models. *NeurIPS*, 2022.

References and further reading

- ▶ Ouyang et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.
- ▶ Wei et al. Emergent abilities of large language models. *TMLR*, 2022.
- ▶ Schaeffer, Miranda, Koyejo. Are emergent abilities of large language models a mirage? *NeurIPS*, 2023.
- ▶ Meta AI. The Llama 3 herd of models. arXiv:2407.21783, 2024.
- ▶ Raschka. *Build a Large Language Model (From Scratch)*. Manning, 2024. Chapters 1–2.